

Neural Network on FPGA

This report summarizes the configuration and source files for the Vivado project file `nn_fpga.xpr`, which implements a 3 Layer Neural Network targeting a Xilinx Artix-7 FPGA.

1. Project Configuration Overview

Parameter	Value	Details
Project Path	<code>nn_fpga/nn_fpga.xpr</code>	Local project file location.
Product Version	Vivado v2024.1 (64-bit)	The version of the Vivado tool used.
Target Part	<code>xc7a200tfbg676-2</code>	A high-density Xilinx Artix-7 FPGA.
Target Board	<code>xilinx.com:ac701:part0:1.4</code>	The Xilinx AC701 Evaluation Board.
Top Module	<code>zyNet</code>	The top-level Verilog module of the design, representing the full network.
Project Type	Default (RTL)	Standard Register-Transfer Level design flow.

2. Design Source Files (`sources_1`)

The design uses a collection of Verilog modules and memory initialization files, characteristic of a hardwired, feed-forward neural network accelerator.

A. Verilog RTL Modules (Neural Network Structure)

The modules define a multi-layered architecture, likely a Fully-Connected (FC) network with custom activation and memory management.

Module Name	Description
<code>zynet.v</code>	The Top-Level Module that integrates all sub-layers and control logic.

Layer_1.v to Layer_4.v	Implementations for the sequential layers of the NN (suggests a 3-layer deep network). Layer 4 is the output layer. Layer 1 = 30 neurons Layer 2 = 30 neurons Layer 3 = 10 neurons Output Layer = 10 neurons
neuron.v	The fundamental computing element, performing the multiply-accumulate (MAC) operation.
relu.v	Implements the Rectified Linear Unit (ReLU) activation function ($f(x) = \max(0, x)$).
maxFinder.v	Module responsible for finding the maximum value in the final output layer (softmax equivalent or final classification).
Sig_ROM.v	Implements the Sigmoid activation function ($\sigma(x)$) using a pre-calculated Read-Only Memory lookup table.
Weight_Memory.v	Handles memory addressing and retrieval for the network parameters (weights and biases).
axi_lite_wrapper.v	Provides an AXI-Lite interface, allowing a controlling processor (e.g., MicroBlaze or Zynq PS) to configure the NN and retrieve results.
include.v	A common practice file for global parameters and definitions.

B. Memory Initialization Files (.mif)

The presence of numerous .mif files confirms that the weights and biases are **hardcoded** into the FPGA's on-chip memory (Block RAMs) during synthesis.

- **Weight Files (w_X_Y.mif)**: 47 files are listed (e.g., w_1_2.mif denotes the weight for 3rd neuron in the 1st layer, and so on).
- **Bias Files (b_X_Y.mif)**: 45 files are listed (e.g., b_1_18.mif denotes the bias for 19th

neuron in the 1st layer).

- **Activation Data (sigContent.mif):** Initialization data for the Sigmoid ROM.

C. Python Files (.py)

- [tf](#) (.py) - This file is used to import the mnist database of handwritten digits and train a sequential tensorflow model where the first layer is a flat layer with 784 inputs and the 3 hidden layers are dense(fully connected) with a 10 neuron output layer. Then we export the weightsandbiases .txt file.
- genWeightsAndBiases(.py) - This takes the txt file and converts the floating point decimal values of weights and biases to fixed point binary representation of datawidth 16 and generates the .mif files.
- genTestData(.py) - This generates 10,000 test samples from the mnist database as test_data_0000.txt till test_data_9999.txt.